

Lecture 17, part 1: Prediction with Abstentions

Thursday, May 28, 2026

Instructors: A. Vijayaraghavan and V. Srinivas

Scribe: N. Reimer

This presentation covered *Beyond Perturbations: Learning Guarantees with Arbitrary Adversarial Test Examples* [Goldwasser, Kalai, Kalai, and Montasser, 2020]. The presentation was given by Huaman Sun.

1 Background

In this paper, our goal and how test examples are drawn differ from classical supervised learning.

Classical supervised learning

- Training examples $(x_i, y_i) \sim P$
- Test examples x also drawn from P
- Goal: learn h with low $\Pr_{x \sim P}[h(x) \neq f(x)]$

This paper

- Training examples: labeled samples from P
- Test examples: arbitrarily drawn from some $Q \neq P$, or selected by an adversary after observing the learned classifier.
- Goal: classify where safe, abstain where training data does not support a reliable label.

We are not assuming the test distribution is close to the training distribution. We want a classifier that abstains when the training data do not support a reliable label. This is an important task because arbitrary test examples break ordinary classification. There are two distinct failure modes arising from arbitrary test examples.

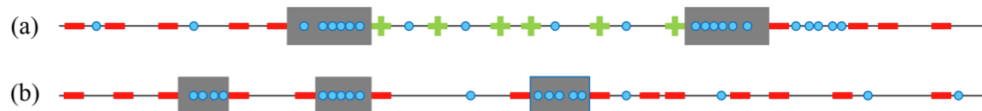


Figure 1: Green + are positive class training examples, red – are negative training examples. Blue points are test examples. Gray regions are areas we want to abstain from predicting on. For this figure, the hypothesis class is intervals.

First, test points may fall in unsupported regions far from all training examples. This results in multiple classifiers in the hypothesis class $\mathcal{H}$ fit to the same training data assigning different labels to the same test point. This corresponds to (a) in 1.

Second, any learned classifier can have a small error region under P . A test distribution or adversary can concentrate on that region. This corresponds to (b) in 1.

2 Setup

Goal

Our goal is to output a selective classifier (h, S) , where h is an ordinary classifier and S is the region of $\mathcal{X}$ where it is safe to classify:

$$h : \mathcal{X} \rightarrow \{0, 1\}, \quad S \subseteq \mathcal{X}.$$

So,

$$h|_S(x) = \begin{cases} h(x), & x \in S, \\ \perp, & x \notin S. \end{cases}$$

S corresponds to all the regions that are not gray in 1.

Learning Objective

Given $h|_S(x)$ the test error under Q is defined as:

$$\text{err}_Q(h|_S) = \Pr_{x \sim Q} [h(x) \neq f(x) \wedge x \in S].$$

where $f : \mathcal{X} \rightarrow \{0, 1\}$ is the true target function. And rejection probability under P is defined as:

$$\perp_P(S) = \Pr_{x \sim P} [x \notin S].$$

The learning objective balances these two terms. A learns $\mathcal{H}$ under arbitrary P, Q if, for every P, Q and every $f \in \mathcal{H}$, with high probability,

$$\text{err}_Q(h|_S) + \perp_P(S) \leq \epsilon.$$

So, because of err_Q we do not want to make too many wrong predictions on the actual test distribution and because of $\perp_P$ we do not want to reject too much of what should be regarded as ordinary, training-distribution data. Essentially, the objective ensures we are accurate on the test distribution while not abstaining from too many examples that are similar to the training distribution.

A natural question arises at this point: why bound $\perp_P(S)$ instead of the rejection rate on the actual test distribution, $\perp_Q(S)$?

The paper emphasizes that absolute rejection under Q cannot be controlled for arbitrary Q . If an adversary or distribution shift places test points in entirely novel regions of the input space, rejecting them is completely unavoidable to maintain accuracy.

However, bounding P -rejection has direct consequences for Q -rejection wherever the distributions Q and P overlap. Specifically, the rejection probability under Q is bounded by the rejection under P plus the Total Variation (TV) distance between the two distributions:

$$\perp_Q(S) \leq \perp_P(S) + \text{TV}(P, Q)$$

Intuitively, if a test point lies in a region where Q is not much larger than P , then controlling rejection under P automatically controls rejection for that part of Q . Conversely, if Q puts mass on entirely new regions (where $\text{TV}(P, Q)$ is large), rejection may be unavoidable.

3 The Redaction Algorithm

The authors then provide an algorithm for constructing S that follows this idea: after training h , search for another classifier $c \in \mathcal{H}$ such that:

1. c agrees with h on the training data;
2. c disagrees with h on many test examples.

That is, c maximizes: (large test disagreement) $- \Lambda \cdot$ (training disagreement).

Intuitively, if c and h both fit the training data but disagree on test points, then the training data does not determine the labels of those test points. Reject where disagreement occurs.

A different idea to address this problem would be to train a classifier to distinguish unlabeled training points from unlabeled test points, and reject points that are clearly test-only. This would be an example of transductive learning: the learner is given the unlabeled test batch before deciding how to classify it.

This would work for test points far from training support; obvious covariate shift; and some out-of-distribution examples.

But it misses dense adversarial clusters near ordinary training points; regions where train/test are not separable, but labels are undetermined; and classifier-specific error concentrations. Algorithm 1 is proposed to address these situations.

Algorithm 1: REDACTION

Require: labeled training data $(\mathbf{x}, \mathbf{y})$, unlabeled test batch $\tilde{\mathbf{x}}$, threshold ϵ , weight $\Lambda = n + 1$

$h \leftarrow \text{ERM}(\mathbf{x}, \mathbf{y})$ $S_1 \leftarrow \mathcal{X}$ $t = 1, 2, \dots$ Choose $c_t \in \mathcal{H}$ maximizing

$$J_t(c) = \text{err}_{\tilde{\mathbf{x}}}(h|_{S_t}, c) - \Lambda \text{err}_{\mathbf{x}}(h, c)$$

$J_t(c_t) \leq \epsilon$ **return** $h|_{S_t}$ $S_{t+1} \leftarrow \{x \in S_t : c_t(x) = h(x)\}$

Each iteration removes/rejects a new set of test points on which a plausible classifier disagrees with h , stopping when it cannot find a c that makes $J_t(c)$ large enough.

Efficient Implementation

So how do we actually choose the maximizing $c \in \mathcal{H}$?

We construct an artificial binary classification dataset and use a standard ERM solver:

- For each training example x_i , we add Λ copies labeled $h(x_i)$. The massive weight Λ heavily penalizes any deviation from h , forcing our new classifier c to anchor itself to h 's training behavior.
- For each test example currently in our safe set S_t , we add one copy with a flipped label: $1 - h(\tilde{x}_j)$. By trying to accurately predict these flipped labels, the ERM naturally maximizes disagreement with h .

Then we want to know that the algorithm will actually terminate quickly. If $J_t(c_t) > \epsilon$, then c_t disagrees with h on more than an ϵ fraction of currently selected test points. Therefore, each successful iteration rejects at least an ϵ fraction of the test batch that had not previously been rejected. Hence:

$$\text{number of iterations} \leq \left\lceil \frac{1}{\epsilon} \right\rceil.$$

VC Class Guarantee

Let $d = \text{VC}(\mathcal{H})$. For any P, Q over $\mathcal{X}$, any $f \in \mathcal{H}$, and sufficiently chosen

$$\epsilon^* = \tilde{\Theta} \left(\sqrt{\frac{d}{n}} \right),$$

The Redaction Algorithm outputs $h|_S$ such that, with probability at least $1 - \delta$,

$$\text{err}_Q(h|_S) \leq 2\epsilon^*, \quad \perp_P(S) \leq \epsilon^*.$$

This holds for arbitrary P, Q without support-overlap assumptions, because the learner is allowed to abstain and has access to unlabeled test samples.

4 Extensions

The Redaction algorithm used labeled training data but the authors also give another algorithm, URedaction that finds the safe regions in an unsupervised manner.

Algorithm 2: UREDACTION without labels

UREDACTION computes the selected set S using only unlabeled training and test examples. More conservative than REDACTION, while it can be paired with any classifier trained separately and can also be used for multiclass learning.

Require: unlabeled training data $\mathbf{x}$, unlabeled test batch $\tilde{\mathbf{x}}$, threshold ϵ , weight $\Lambda = n + 1$

$S_1 \leftarrow \mathcal{X}$ $t = 1, 2, \dots$ Choose $c_t, c'_t \in \mathcal{H}$ maximizing

$$J_t(c, c') = \text{err}_{\tilde{\mathbf{x}}}(c|_{S_t}, c') - \Lambda \text{err}_{\mathbf{x}}(c, c').$$

$J_t(c_t, c'_t) \leq \epsilon$ **return** S_t $S_{t+1} \leftarrow \{x \in S_t : c_t(x) = c'_t(x)\}$

The paper also studies settings where realizability fails ($f \notin \mathcal{H}$). Massart noise

- Each example may have label noise rate $\eta(x) \leq \eta < 1/2$.
- The algorithm can still work, but requires more samples and modified analysis.

Semi-agnostic setting

- There exists some $h^* \in \mathcal{H}$ with error at most η on both training and test distributions.
- Even then, one cannot generally guarantee error and rejection below order $\sqrt{\eta}$.
- REDACTION nearly achieves this limitation.

References

Shafi Goldwasser, Adam Tauman Kalai, Yael Tauman Kalai, and Omar Montasser. Beyond perturbations: Learning guarantees with arbitrary adversarial test examples. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL <https://arxiv.org/abs/2007.05145>.